Intrusion Detection System (IDS)

Uriel Shapiro, Guy Shimon

December 2024

Contents

1 Introduction **2**

2 System Flow **3**
2.1 Packet Processing 3
2.2 Storing Results 3

3 Architecture **4**
3.1 IDS Class 4
3.2 FlowAnalyzer Class 6
3.3 PacketAnalyzer Class 7
3.4 FlowAbnormality Class 12
3.5 FlowAbnormalityDB Class: 14

4 Research Process **16**

5 References **18**

1 Introduction

In this report, we present the the thoughts and process behind our Intrusion Detection System (IDS) for detecting data exfiltration attempts. This system was developed as the final project for the *"Communication Protocols Protection"* course at Ariel University.

The primary objective of the system is to identify and alert on potential data exfiltration attempts from within an organization, whether these attempts are successful or unsuccessful. The system achieves this by monitoring and analyzing network traffic for protocol abnormalities, providing a robust mechanism for detecting suspicious activities.

Our approach focuses on two levels of analysis:

- **Protocol-level abnormalities:** Identifying irregularities within individual packets by examining adherence to expected protocol behaviors.
- **Flow-level abnormalities:** Analyzing sequences of packets within a flow to detect patterns indicative of potential exfiltration.

The system operates in a detection-only capacity, raising alerts upon identifying abnormalities and stores the abnormalities for statistics.

2 System Flow

The system operates by allowing the user to choose one of two methods for packet collection:

- **Sniffing packets** directly from a network interface.
- **Reading packets** from a pre-existing `.pcap` file.

If the user selects the sniffing option, they are prompted to choose the network interface from which packets will be captured. Alternatively, if the user opts to read packets from a file, they are prompted to provide the file path to the `.pcap` file.

2.1 Packet Processing

Once packets are collected, they are stored in a queue, which is processed by a separate thread. This thread handles the following tasks:

1. **Flow Association:** Each packet is categorized into its corresponding flow based on its five tuple (source IP, destination IP, source port, destination port, protocol).
2. **Single-Packet Analysis:** The packet is analyzed for abnormalities in each of its headers.
3. **Flow-Level Analysis:** The packet is also examined in the context of the entire flow to detect abnormalities across a sequence of packets.

Whenever an abnormality is identified, it is logged in the flow's abnormality list.

2.2 Storing Results

After the packet collection process (sniffing or reading) is complete, all identified abnormalities across all flows are compiled and stored in a local SQL database. This database allows the user to:

- Retrieve detailed logs of detected abnormalities.
- Generate statistics for further analysis.

This approach not only facilitates efficient data retrieval but also enhances the system's modularity. By using an SQL database, the system can be easily extended to support additional queries in the future without requiring significant modifications to the existing code.

3 Architecture

3.1 IDS Class

The `IDS` class serves as the central API to the entire intrusion detection system. It acts as a facade, providing a unified interface for initializing, managing, and interacting with the system's core functionalities. Regardless of changes to the underlying methods in the auxiliary classes it depends on, such as `PacketSniffer`, `FlowAnalyzer`, and `FlowAbnormalityDB`, the `IDS` class encapsulates the complexity of these components and ensures a consistent and simplified interface for the user.
Below is a detailed description of its architecture and functionality:

Responsibilities

The `IDS` class is responsible for:

- Initializing the system based on the selected mode (sniffing packets or reading from a `.pcap` file).
- Managing multithreading to ensure efficient packet handling and analysis.
- Storing packet abnormalities in a structured database for further analysis.
- Managing the user interface and coordinating the invocation of specific handlers to manage various operations, such as database interactions and logging.

Attributes

The `IDS` class includes the following key attributes:

- `snifferFlag`: Boolean flag indicating whether to sniff packets or read them from a pcap file. Is set when an IDS instance is created.
- `packet_queue`: A thread-safe queue for storing packets awaiting analysis.
- `cond`: A `Condition` variable to synchronize threads.
- `flow_analyzer`: An instance of the `FlowAnalyzer` class to handle packet flow abnormalities.
- `db`: An instance of the `FlowAbnormalityDB` class for storing abnormalities in an SQL database.
- `logger`: A logging object for recording events and debugging information.

Methods

- `__init__(self, sniff_flag, sniff_interface=None, pcap_path=None)`: Initializes the IDS, setting up the logger and selecting the data collection method (sniffing or reading from a file).
- `start(self)`: Launches the IDS by starting the appropriate threads for packet collection and analysis.
- `stop(self)`: Stops the IDS operation, ensures threads are joined, and inserts detected abnormalities into the SQL database. Then prompts the user to send SQL queries using a specific handler.
- `sniff_packets(self)`: Continuously sniffs packets from the selected interface, adds them to the packet queue, and notifies waiting analyzer thread for each sniffed packet.
- `read_pcap(self)`: Reads packets from a provided `.pcap` file, adds them to the packet queue, and notifies waiting analyzer thread for each packet.
- `analyze_packet(self)`: Processes packets from the queue, checks for abnormalities, and updates the flow analyzer with detected abnormalities.
- `setup_logger(self)`: Configures the logging system to record IDS events with appropriate formatting, settings and directory.
- `__del__(self)`: Ensures the database connection is closed when the IDS instance is deleted.

Multithreading and Synchronization

The IDS operates with two primary threads:

1. **Packet Collection Thread**: Responsible for either sniffing packets or reading them from a pcap file.
2. **Packet Analyzer Thread**: Processes packets, identifies abnormalities, and updates the flow analyzer.

Synchronization between these threads is achieved using a `Condition` variable, ensuring efficient handling of packets and coordinated stopping of operations.

Extensibility and Modularity

The `IDS` class is designed with modularity in mind:

- The flow analyzer and database components are encapsulated, allowing for easy replacement or extension.
- The SQL-based storage system supports adding new queries or reports without modifying the core analysis logic.
- Threading and queue-based architecture ensures the system can handle high volumes of traffic with minimal bottlenecks.

3.2 FlowAnalyzer Class

The `FlowAnalyzer` class is responsible for managing and analyzing all network flows within the IDS. A flow is defined by its five-tuple: source IP, destination IP, source port, destination port, and protocol. This class serves as a centralized storage for flows, providing methods for updating, adding, and identifying abnormalities in network flows.
The class holds a dictionary of flows, where the keys are five tuples of packets and the values are `PacketAnalyzer` instances. Where each instance updates with each `update_packet` call.

Responsibilities

- Maintain a mapping of flows identified by their five-tuples.
- Update existing flows with new packets or log errors when attempting to update non-existent flows.
- Add new packets to create flows and associate them with a `PacketAnalyzer`.
- Provide access to detected abnormalities by iterating through all flows.
- Generate summaries of flow statistics, including the total number of packets, data transferred, and abnormalities.

Key Features

- `__contains__(self, packet)`: Check whether a given packet belongs to an existing flow, considering both forward and reverse directions.
- `update_flow`: Update the state of an existing flow with a new packet, ensuring accurate tracking.
- `add_packet`: Add a packet to the flow storage, initializing a new flow when necessary.
- `get_abnormalities`: Yield abnormalities detected in flows, paired with their corresponding five-tuples.
- `print_flows`: Print flow statistics, including total packets, data transferred, and specific abnormalities for each flow.

3.3 PacketAnalyzer Class

The `PacketAnalyzer` class is the core logic component for analyzing network packets and detecting abnormalities within the IDS. It utilizes the Scapy library to parse packets and extract relevant information for flow and anomaly detection. This class tracks metadata for each packet, identifies protocol-specific behaviors, and records abnormalities.

Responsibilities

- Parse and analyze packets to extract key details such as five-tuple and MAC addresses.
- Maintain protocol-specific state information. Is used for flow level analysis.
- Detect and log abnormalities in the network traffic, including failed authentications, suspected DNS tunneling, and other protocol misuses and abnormalities.
- Aggregate packet-level statistics such as total data transferred, number of packets, and resolved IP addresses or requested domains.
- Interface with the `FlowAnalyzer` class by serving as the analytical unit for each individual flow.

Key Features

- `__init__`: Initializes a new packet analysis instance, extracting metadata from the provided packet and analyzing it.
- `update_packet`: Updates the current analysis with a new packet, aggregating statistics and identifying additional abnormalities.
- `analyze_packet`: Processes individual packets based on their protocol, identifying and recording specific anomalies.

Network Protocols Processed

The following protocols are processed by the IDS during packet analysis:

- **TCP Protocol**:
  - The system checks for anomalies related to TCP flags and ensures correct flag combinations.
  - It detects the TCP three-way handshake and verifies that SYN, SYN ACK, and ACK flags are set correctly and by order.
  - The system identifies potential Denial of Service (DoS) attacks.
- **SSH Protocol**:

- The system monitors for SSH packets, ensuring they are on the standard port (22).
- It tracks failed login attempts using the TCP & RST flag and detects potential brute-force attacks if failed attempts exceed a set threshold within a given time window.
- Large payload sizes in SSH packets are also flagged as anomalies.

- **HTTP Protocol**:
  - The system performs several checks on both incoming HTTP requests and responses, raising alerts when certain conditions are violated.
  - The system verifies if the destination IP address of the packet exists in the list of resolved IP addresses. If the IP address is not in the list, is is flagged as an anomaly.
  - The system checks if the source or destination port of the packet is 80, which is the standard port for HTTP traffic. If neither port matches 80, an anomaly is logged.
  - If the packet contains an HTTP request (layer `HTTPRequest`), several checks are performed on the request:
    1. **Missing Method or Host:** If either the `Method` or `Host` fields are missing, an anomaly of type `HEADER_VIOLATION` is triggered.
    2. **Unusual HTTP Method:** The method used in the HTTP request is checked against a set of valid methods (`GET, POST, HEAD, PUT, DELETE, OPTIONS, PATCH`). If the method is not in the list, a `WARNING` anomaly is raised.
    3. **Overly Long URL:** If the URL (represented by `Path`) is longer than 2000 characters, a `PAYLOAD_VIOLATION` anomaly is raised.
    4. **Missing User-Agent Header:** If the `User-Agent` header is absent in the request, a `HEADER_VIOLATION` anomaly is raised.
  - If the packet contains an HTTP response (layer `HTTPResponse`), the function performs the following checks:
    1. **Unusual Status Code:** The HTTP status code is validated to be in the range 100-599. If the status code is outside this range, an anomaly of type `WARNING` is logged.
    2. **Overly Long Response:** If the response size exceeds certain amount of bytes, a `PAYLOAD_VIOLATION` anomaly is raised.

- **FTP Protocol**:
  - The system monitors FTP traffic and flags potential brute-force attacks based on the frequency of failed login attempts.

- **SMB Protocol**:

  - The system processes SMB packets, checking for session setup failures and potential brute-force attacks based on error responses.
  - It flags suspicious access to administrative shares such as `C$`, `ADMIN$`, and `IPC$`.

- **IMAP Protocol**:
  - The system processes IMAP packets, particularly monitoring login attempts and command flooding.
  - Failed login responses and multiple rapid commands are flagged as potential anomalies.

- **POP3 Protocol**:
  - The system monitors POP3 traffic and checks for authentication failures and data exfiltration attempts based on repeated commands or large numbers of retrieved messages.

- **UDP Protocol**:
  - The system processes UDP packets and checks for specific protocols such as DNS and DHCP.

- **DNS Protocol**:
  - A check for abnormal DNS header flags, ensuring that the flags and fields in the DNS header are valid is done.
    1. Validates the DNS QR flag to ensure it is either 0 (query) or 1 (response).
    2. Checks the DNS opcode to ensure it is within the valid set of supported opcodes.
    3. Verifies that the DNS RCODE is within the acceptable range.
    4. Ensures that flags such as AA, TC, RD, and RA are set to valid values (values other then 0 or 1 are indications of protocol misuse).
  - **DNS Query Processing:**
    1. If the packet is a DNS query, the domain name is extracted and checked for potential tunneling or excessive queries.
    2. The domain name is extracted and simplified using the `tldextract` library.
    3. If the domain name has been previously requested, a potential DNS tunneling anomaly is raised.
    4. A check is performed for DNS packet ID reuse (transaction ID).
    5. If the query length exceeds a predefined threshold, an anomaly is raised.
  - **DNS Response Processing:**

1. If the packet is a DNS response, it is verified that the corresponding query exists in the flow and that the query and response originate from the same IP address.
2. The resolved IP addresses from the DNS response are added to the list of resolved IPs.
3. Transaction ID matching is checked, and anomalies are raised for mismatches or reused IDs (can be an indication of Kaminsky attack).
4. Specific checks are made for mDNS packets, such as ensuring the authoritative flag is set and that the packet is sent to the standard multicast address.
5. If the DNS response does not meet the standard conditions (e.g., opcode, query count), anomalies are raised.
6. A check is made for abnormal packet sizes exceeding a certain amount of bytes.

- **DHCP Protocol**:
  - The system processes DHCP packets, checking for abnormalities such as invalid ports, missing message types, reused transaction IDs, and issues with DHCP options.
  - The checks the system does:
    1. **Port Validation:** The method checks whether the packet is on one of the standard DHCP ports (67 or 68).
    2. **Message Type Validation:** The method ensures that the message type is present and valid (e.g., DHCP Discover, Offer, Request).
    3. **Transaction ID Validation:** Checks for transaction ID reuse and ensures it is not already in the list.
    4. **Hardware Address Length:** Ensures that the hardware address length is 6 bytes, which is standard for Ethernet.
    5. **Broadcast Flag Validation:** Checks if the broadcast flag is set to one of the valid values (0x0000 or 0x8000).
    6. **Option Count Check:** If the DHCP options count exceeds 20, an anomaly is raised.
    7. **Server Identifier Validation:** Ensures that the server identifier is present in DHCP Offer or Ack messages.
- **MAC Protocol**:
  - Two types of anomalies are detected:
    1. **ARP Spoofing:** Multiple MAC addresses are associated with a single IP address.
    2. **IP-MAC Mismatch:** The observed MAC address does not match the expected MAC address for a given IP address.

- **ICMP Protocol (using IPv4):**
  - The system inspects ICMP packets for various anomalies:
    1. **Unusual ICMP types:** Abnormal ICMP types are flagged.
    2. **Redirect messages (Type 5):** ICMP redirects are detected and flagged as potential security issues.
    3. **Ping Amplification:** If an ICMP Echo Request (Type 8) is sent to a broadcast address, it is flagged as potential amplification.
    4. **Oversized packets (ICMP tunneling):** Packets larger than a set threshold (100 bytes) are flagged as possible ICMP tunneling.
    5. **Traceroute detection (TTL exceeded, Type 11):** If ICMP type 11 is detected, it may indicate a traceroute attempt.
    6. **Echo Reply without Request:** If an ICMP Echo Reply (Type 0) is received without a corresponding Echo Request, an anomaly is flagged.
    7. **Destination Unreachable:** ICMP types 3 and 4 indicate destination unreachable errors and may signal flooding or scanning activity.
- **ICMP Protocol (using IPv6):**
  - The system inspects ICMPv6 packets for various anomalies:
    1. **Unusual ICMPv6 Types:** Abnormal ICMP types are flagged
    2. **High Traffic Rate:** The function monitors the rate of ICMPv6 packets originating from the same source.If the rate exceeds a predefined threshold within a specified timeframe it raises a warning for potential flood or scanning activity.
    3. **ICMPv6 Redirect Messages:**
       Redirect messages (layer `ICMPv6ND_Redirect`) are checked, and anomalies are flagged if they exceed acceptable thresholds. These messages could indicate potential man-in-the-middle (MITM) attacks or misconfigurations.
    4. **Ping Amplification:**
       The function detects ICMPv6 Neighbor Solicitation (NS) packets sent to multicast addresses (e.g., `ff02::1`). Such behavior may indicate a potential amplification attack leveraging broadcast requests.
    5. **Oversized Packets (ICMPv6 Tunneling):** Packets exceeding a specific size limit are flagged as potential ICMPv6 tunneling attempts.
    6. **Traceroute Detection:**
       Time Exceeded messages (`ICMPv6TimeExceeded` layer) are monitored to detect potential traceroute activities. Such activities could indicate network reconnaissance or misconfigured TTL values.

Metadata tracking:

- Tracks the total number of packets and the cumulative size of data transferred.
- Maintains protocol-specific logs and timestamps for abnormal events.
- Stores resolved IP addresses and requested domain names for further analysis.

3.4 FlowAbnormality Class

The `FlowAbnormality` class serves as a foundational component for representing and categorizing network abnormalities detected by the system. Each flagged abnormality is instantiated as an object of this class, encapsulating details such as its type, description, and severity level.

Class Structure

The `FlowAbnormality` class is defined with the following attributes and methods:

- **Attributes:**
  - `abnormality_type:` A string representing the category or specific type of abnormality (e.g., ICMPv6 Abnormality, TCP Abnormality, etc.).
  - `description:` A detailed description of the abnormality.
  - `level:` An enumerated value (`AbnormalityType`) representing the severity of the abnormality.
- **Methods:**
  - `get_level():` Returns the severity level of the abnormality as a string.
  - Equality (`__eq__`) and hashing (`__hash__`) operators: Facilitate comparisons and usage of the class in data structures such as dictionaries and lists (The abnormalities are stored in a list in each PacketAnalyzer instance).

Abnormality Types

The severity levels for abnormalities are managed using the `AbnormalityType` enumeration, which provides a structured approach to classify abnormalities. Each type reflects the nature and criticality of the detected issue. Key severity levels include:

- **INFO:** Informational messages about non-critical observations.

- **WARNING:** Signals potentially suspicious activity that may require attention.
- **FLAGS_VIOLATION:** Indicates issues with protocol flag settings.
- **HEADER_VIOLATION:** Reflects abnormalities in the packet header structure.
- **PAYLOAD_VIOLATION:** Flags issues with the packet's payload, such unusual size of payload.
- **ALERT:** The most severe indication. indicating of high-severity abnormalities.
- **PORT_VIOLATION:** Highlights irregularities related to port usage.
- **TRANSACTION_VIOLATION:** Represents abnormalities in protocol-specific transactions (reused Transaction ID for example).

3.5 FlowAbnormalityDB Class:

The `FlowAbnormalityDB` class serves as a local SQL database handler for storing and managing flagged network abnormalities detected by the IDS. This class uses SQLite for efficient data storage and retrieval.

Class Structure

The `FlowAbnormalityDB` class is designed with the following attributes and methods to handle database operations seamlessly:

- **Attributes:**
  - `db_name`: The default database name.
  - `connection`: An active SQLite connection object.
  - `cursor`: A cursor object for executing SQL queries.
  - `logger`: A logging instance configured for the class to provide detailed runtime information.
- **Methods:**
  - `__init__()`: Initializes the database connection, configures logging, and creates the required database table.
  - `create_table()`: Creates a table named `flow_abnormalities` to store records of detected abnormalities.
  - `insert_abnormality()`: Inserts a `FlowAbnormality` instance along with relevant metadata (source, destination, ports, protocol) into the database.
  - `get_total_statistics()`: Retrieves aggregate statistics for each abnormality type and severity level.
  - `get_distinct_sources()`: Fetches all unique source addresses from the database.
  - `get_all_abnormality_types()`: Retrieves all distinct abnormality types stored in the database.
  - `execute_custom_query()`: Allows the execution of custom SQL queries with optional parameters.
  - `fetch_all_abnormalities()`: Fetches all records stored in the `flow_abnormalities` table.
  - `close()`: Closes the database connection.

Database Schema

The `flow_abnormalities` table is structured as follows:

- `id`: A unique identifier (primary key) for each abnormality record (an ascending index).
- `src`: The source IP address of the packet.
- `dst`: The destination IP address of the packet.
- `src_port`: The source port number.
- `dst_port`: The destination port number.
- `protocol`: The protocol used.
- `abnormality_type`: The category of abnormality (e.g., Flags Violation, Header Violation).
- `description`: A detailed explanation of the abnormality.
- `level`: The severity level (e.g., INFO, WARNING, ALERT).

4 Research Process

As we developed this project, we conducted a detailed analysis of each protocol to establish a baseline for their normal behavior. This work laid the groundwork for identifying unusual patterns that might signal data exfiltration attempts. By gaining a deep understanding of how each protocol typically operates—including their subtle nuances—we were able to design customized detection mechanisms to effectively pinpoint these irregularities.

Although the techniques used to detect protocol abnormalities have already been described, we want to note a few key methodologies:

1. **check_dos(packet):** This method processes a TCP packet to identify potential SYN flood or DoS attacks. While these attacks may not be directly associated with data exfiltration, they can serve as a diversion or cover for exfiltration activities conducted over standard protocols..

2. **check_times(buffered_list, src_ip, delta, threshold, ...):** This method evaluates whether a specific number of packets have been transmitted by the same IP address within a defined time window. Such behavior may indicate potential data exfiltration at the flow level.

3. **TCP 3-Way Handshake:** This method monitors the establishment of a TCP connection by analyzing the sequence of packets in the 3-Way handshake process. If the handshake has not yet been completed for a given flow, the packet flags are inspected to identify the relevant flags required at each stage of the handshake. Once a full 3-Way handshake is established, the system further verifies that not all packets are from the same sender, and that the souce IP of each packet is legit (SYN sender and SYN-ACK sender are the same, etc.).

4. **POP3, IMAP, and SMB Protocol Processing:** The methods to detect abnormalities in these protocols are designed to analyze traffic related to the POP3, IMAP, and SMB protocols, focusing on detecting potential anomalies associated with data exfiltration and brute force attacks. Each method examines the raw payload of the incoming packet to detect specific patterns indicating abnormal behavior. For instance, in POP3, failed authentication attempts are flagged when multiple "incorrect" responses follow repeated "USER" and "PASS" commands, while command flooding and data exfiltration are identified through excessive "RETR" commands. Similarly, for IMAP, failed login attempts and rapid command execution (such as multiple "SELECT", "FETCH", or "LOGIN" commands) are indicators of brute force and DoS attacks. In the SMB protocol, failed authentication is detected through the "STATUS_ACCESS_DENIED" response, while suspicious access to administrative shares, such as "C$", "ADMIN$", or "IPC$", is flagged for further analysis. For all three protocols, the `check_times` method is utilized to track the frequency of abnormal events within a given time window, allowing for the detection of

potential brute force attempts and DoS attacks. These methods share a common structure in their approach to anomaly detection, leveraging time-based thresholds and predefined criteria to identify and flag suspicious activity across these protocols.

5. **SSH Protocol Processing:** The method that processes SSH packets checks for standard SSH protocol behavior, and afterwards examines TCP reset (RST) flags, which may indicate failed login attempts. Additionally, the method checks for large payloads using the TCP PSH and ACK flags, which could indicate suspicious activity such as data exfiltration.

6. **DNS Protocol Processing:** DNS packets are analyzed for potential anomalies such as DNS tunneling, DNS packet ID reuse, and abnormal packet sizes. For query packets, the method monitors repeated domain requests, checks for unusually long query names, and detects potential DNS tunneling by comparing domain names. It also tracks failed queries and checks the validity of DNS IDs. For response packets, it verifies that the response matches an earlier query, checks for DNS packet ID mismatches, and flags unauthorized mDNS behavior. Suspicious DNS query and response patterns, such as a high number of questions or answers, are also detected. The system flags DNS packets with abnormal packet size.

7. **DHCP Protocol Processing:** The system processes DHCP packets by performing checks for valid ports, message types, transaction ID reuse, hardware address length, broadcast flag, suspicious number of options, server identifier in OFFER/ACK, and requested IP in REQUEST.

8. **HTTP Protocol Processing:** The system processes HTTP packets by checking if the destination IP is in the resolved IP list, ensuring the port validation, validating HTTP request methods, headers (such as missing 'User-Agent' field in request header), and URLs, and inspecting HTTP response status codes and lengths.

9. **MAC Protocol Processing:** The system checks for IP-MAC mismatches by verifying ARP replies for conflicting MAC addresses associated with an IP (indicating possible ARP spoofing) and validating the consistency of IP-MAC mappings in Ethernet traffic.

5 References

- Scapy - Used for packet level analysis.
- humanfriendly - Used for human friendly statistics printing.
- tldextract - Used to extract relevant data from URLs when DNS packets are analyzed.